DAWN

AN AI CODING PARTY YOU COMMAND
FROM YOUR POCKET

THE PITCH

DAWN is a team of AI coding agents you lead like an RPG party: send them on quests from your phone, approve their moves in dialogue boxes, and collect the loot (working code).

My own take on Cursor / Google Antigravity, built by me with AI coding agents. The AI is called **DAWN**. The interface is a retro pixel RPG.

HOW THE GAME MAPS TO THE WORK

PARTY MEMBERS

Agents. Each has a **class** (role: Builder, Fixer, Tester, Designer) and a model behind it.

HP · LEVEL

HP = budget left (money/tokens). **Level** = experience: quests finished on this repo.

QUESTS

Tasks. "Add a dark mode." "Fix the login bug." One sentence starts one.

DIALOGUE BOXES

Approvals. *"DAWN wants to edit 3 files. YES / NO"*, with the changes (the **diff**) shown.

LOOT

Pull requests (**PRs**): a bundle of finished changes, ready for me to merge into the project.

BATTLES

Errors and failing tests. The party fights them; I can retry, swap a member, or flee.

OVERWORLD MAP

The project. Folders are regions, key files are towns; you can see where your party is working right now.



MOCKUP · MVP 1 QUEST SCREEN
(ORIGINAL ART, HTML/CSS)

THE TWO MVPs

MVP = SMALLEST VERSION WORTH USING

MVP 1 · MOBILE CONSOLE (iOS FIRST)

Job: start a quest from my phone, watch the agents work live, approve or reject their changes, and see the result, without opening a laptop.

PICK: REACT NATIVE + EXPO
Expo is a toolkit that lets one TypeScript codebase run as an iPhone app. Why over SwiftUI (Apple's native toolkit): AI coding agents are strongest in TypeScript, I can test on my phone instantly via the Expo Go app without Xcode builds, Android comes later almost free, and MVP 2 uses the same language.

How it fits together

PHONE
Expo app. Sends quests, shows the live log, YES/NO

⇄

DAWN SERVER
Small Python app on **Cloud Run** (Google's rent-by-the-second servers)

➡

AGENTS
Antigravity SDK runs the agent loop + approval pauses

➡

GITHUB
Clone repo, branch, push, open PR (loot)

- **Agent engine: Google Antigravity SDK.** `pip install google-antigravity`, Python, Apache-2.0 license, built on Gemini. *An SDK is a kit of ready-made code; this one already handles tools, sessions, safety policies, subagents and human-approval steps.*
- **Model router: OpenRouter.** *One account and API that reaches many AI models.* Lets party members run on OpenAI, Grok, Claude, or open-weight models (Gemma, Llama, Qwen, gpt-oss) alongside Gemini.
- **Live updates:** server streams agent events to the phone (WebSocket = an always-open line). **Approvals** pause the agent until I tap.

Key screens (MVP 1)

- **Title / Start**: sign in, connect GitHub, pick a repo.
- **Overworld**: the repo as a map; shows active quests and where agents are.
- **Quest Board**: type a new quest, pick the party, see queued and finished quests.
- **Party**: each agent's class, model, HP (budget left) and level.
- **Quest Live**: streaming log as RPG text; battles appear when errors hit.
- **Dialogue + Loot**: approve/reject the diff, then view the PR and merge.

MVP 2 · MAC DESIGN TOOL

Job: see my web app live, **click any element** (a button, a heading), type "make this bigger and orange," and an agent edits the real code. The preview refreshes on its own.

Path: extension first, fork later

- **Step 1: a VS Code extension** *(an add-on that runs inside the editor)*. A preview panel injects a tiny click-to-select script, sends the element + my request to DAWN, the agent edits the file.
- Publish on **Open VSX** *(the open extension store)* so it works in Cursor, Antigravity and other VS Code–based editors.
- **Why not fork Antigravity?** It is VS Code–based but closed-source, so it can't be copied and modified.
- **Step 2, only if needed: fork Code-OSS** *(VS Code's open-source core, MIT license; a fork = my own copy to change)* for a fully custom DAWN editor.
- **Fork rule:** forks must use Open VSX, not Microsoft's marketplace (its terms limit it to Microsoft's own products).

WHY THIS ORDER

- MVP 1 builds the core everything else needs: the **DAWN server** that runs agents.
- MVP 2 reuses that same server; it is "just" a new controller.
- The phone app is the most "DAWN" part: the RPG feel shows up first.

THE ROADMAP

REALISTIC PACE · NIGHTS + WEEKENDS

STAGE 1

FIRST WEEKEND

RUNNING BY SUNDAY

- Repo `dawn` on GitHub; a test repo to practice on.
- One Antigravity agent runs on my laptop and edits the test repo.
- Same server deployed to Cloud Run.
- Expo app on my phone: type a quest, watch the log stream in.
- Stretch: one YES/NO approval box.

STAGE 2

WEEKS 2–4

USABLE BY ME

- GitHub login; each quest makes a branch and a PR (loot).
- Approval dialogue with a readable diff.
- Party + HP: per-quest budget caps that stop the agent.
- OpenRouter: add 1–2 non-Gemini party members.
- Push notifications; first **TestFlight** build *(Apple's beta-testing app)*.

STAGE 3

MONTHS 2–3

FEELS LIKE A GAME

- Multi-agent parties (subagents), levels and XP.
- Overworld map of the repo; battle screen for errors.
- Pixel art + sound pass; polish onboarding.
- MVP 2 prototype: VS Code extension with click-to-edit preview.
- Decide: App Store, or keep it private.

MONTHLY COSTS

ESTIMATES

ITEM	PER MONTH	ASSUMPTION / SOURCE
Apple Developer Program	**≈ $8**	Known fact: $99/year (needed for TestFlight + App Store).
Cloud Run server	**$0–20**	Just me, a few agent hours a day. Free tier covers the first 180,000 vCPU-seconds (≈50 CPU-hours) a month on request-billed services.
AI model usage	**$10–200**	The big variable. Low end: a few quests a day on cheaper or free-tier models. High end: daily heavy use of top models. OpenRouter adds a 5.5% fee when buying credits.
GitHub · Expo	**$0**	Free tiers are enough to start.
Total to start	**≈ $20–230**	**Set hard spending caps on day one (that's what HP is for).**

Prices checked Oct 2026 (apple.com, cloud.google.com/run/pricing, openrouter.ai/pricing). Model prices change often; recheck before scaling.

RISKS + OPEN QUESTIONS

- **SDK is young and Gemini-first.** v0.1.x, and it ships a compiled program inside (separate Mac/Linux/Windows builds). Plugging in OpenRouter models is unproven; keep a thin adapter so the engine can be swapped.
- **Model costs** can spike when an agent loops. Budget caps + "flee" button are must-haves.
- **App Store review:** Apple limits apps that run code. DAWN runs code on the server, not the phone, but expect questions; stay on TestFlight until it's solid.
- **Long quests vs. server limits:** Cloud Run requests time out (max 60 min); long jobs may need a background worker.
- **Name check:** "DAWN" is a common word. Search trademarks (USPTO) and the App Store before branding.

DAWN: Create repo dawn and start MVP 1 with a coding agent?

First quest for the agent: "Set up a Python server using the Antigravity SDK that runs one agent on a test repo and streams its log; plus an Expo app that sends a quest and shows the stream."

▶ YES NO